

Dragon Ball Super Fighters: Godot Game Plan

This document outlines the detailed plan for "Dragon Ball Super Fighters", a 2.5D anime-inspired fighting game, designed to be achievable by a beginner within a strict 20-hour development timeline in Godot.

1. Game Overview

- Title (Working): Dragon Ball Super Fighters
- Engine: Godot v4.4.1
- Genre: 2.5D Anime-Inspired Fighting Game. This means using 3D models and environments, but with fighting strictly on a 2D plane.
- Total Development Time: A strict 20 hours total, spread over 9 days (~2-3 hours per day).
- Target Visual Style: To look like Guilty Gear Strive or Dragon Ball FighterZ – anime-style 3D models on a 2D plane with cel-shaded graphics, thick black outlines, and vibrant colours. The game will not be pixelated.
- Beginner-Friendly: Designed for a beginner with Python basics, new to GDScript. No custom graphics or animations are needed.
- Goal: To deliver a complete, playable game that feels professional and polished within the given timeframe.

2. Core Gameplay Features

- Perspective: Side-on 2.5D view, with the camera locked to a side-on angle, similar to Dragon Ball FighterZ. The camera will zoom slightly when super moves are activated.
- Characters (Minimum 3):
 - Roku (Goku-inspired, white hair):
 - Playstyle: Balanced. Easy to use, versatile, no clear specialty.
 - Super Move: Kamehameha (a charge beam that pierces). It will have a peak cinematic cutscene.
 - Combos:
 - Basic: Punch → Punch → Blast.
 - Complex: Kick → Punch → Up Smash → Energy Shot.
 - Blackza (Frieza-inspired, black skin with glowing purple lines):
 - Playstyle: Tricky / Zoner. Harder to use, ranged attacks, stuns, weak up-close. Will be slightly stronger than balanced characters.
 - Super Move: Death Ball Counter (cuts through Kamehameha, slow charge, massive AOE). Will have a peak cinematic cutscene.
 - Combos:
 - Basic: Tail Swipe → Hover Dash → Beam Shot.
 - Complex: Teleport → Punch → Tail Slam → Blast Ring.
 - Vegeta:
 - Playstyle: Technical. Requires timing & control, but hits harder. Will be

slightly stronger.

- Super Move: Final Flash (charge up – straight-line blast). Will have a peak cinematic cutscene.
- Combos:
 - Basic: Jab → Knee Strike → Ki Burst.
 - Complex: Dash Cancel → Uppercut → Blast Barrage → Smash Kick.
- Controls: Basic input for movement (WASD/arrow keys), attack (light/heavy), dash/evade, charge Ki, and 1-2 super moves.
- Combat Mechanics:
 - Light and Heavy Attacks: Yes, aiming for light and heavy attacks/fun varieties if achievable, with the choice of achievability being decided by me.
 - Combo System: Chained input-based combos (pressing buttons in sequence). Each character will have a basic combo and a more complex combo for extra damage.
 - Super Moves: Each character has at least 1-2 super moves with a dedicated cutscene.
 - Power-up System: If time allows, a power meter can build from combos or charging, triggering supers (1-2 max per match). Power orbs could randomly spawn to fill Ki or heal.
- End Screen & Replay: Yes, an end screen and replay screen are essential.
- Character Change: Yes, character change functionality is essential.

3. Arena & Environment

- Environment: A single Tournament of Power-style arena.
- Style: A flat floating platform with cracked tiles and scattered debris.
- Background: A cosmic space skybox with purple/blue nebula clouds and distant stars.
- Dynamic Effects: Debris floats up during super moves. Cracks, dust, and shockwaves are desired during super moves if achievable.

4. Visuals & Presentation

- Graphics: "Peak" anime-style visuals using cel-shaded 3D models with thick black outlines. This style will resemble Guilty Gear or Dragon Ball FighterZ. No pixelation or "AI-generated" look.
 - Bloom: Use Godot's Glow post-processing to create a bright fringe or soft glow around powerful beams and impacts.
 - Particle Effects (VFX): Godot's Particles2D/3D will be key. Use for Ki charge glows, explosions, aura flares, and energy beams/spheres.
 - Motion Polish: Include camera shake on hits, slow-motion moments when super moves land. Use motion trails on fast moves.
- Cutscenes: "Peak" cinematic sequences for super moves. They don't have to be short, but they must look good. Each character will have unique winning cutscenes.

- Elements: Dynamic camera angles (zooms, swings, multi-angle shots), screen flashes, slow-motion bursts, particle effects, dramatic lighting, and character-specific reactions.
- Implementation: Achieved using Godot's AnimationPlayer, Tween animations, and Camera2D/3D. No drawing frames by hand.
- Text Effects: Cool, animated text for various in-game events.
 - Startup: "FIGHT!". For the title, "DRAGON BALL SUPER FIGHTERS" in Saiyan Sans with a glowing aura effect and scaling in.
 - KO Screens: "KNOCKOUT!" in Big Noodle Titling or Anton, with screen shake and fade-out.
 - Super Moves: Text like "KAMEHAMEHA!!!" in Saiyan Sans, with motion blur and particle sparks.
 - Fonts: Bold, powerful anime-style fonts are key. Recommended fonts include Saiyan Sans, Big Noodle Titling, Anton, SF Fedora Heavy, and Bangers.
- User Interface (UI):
 - Start Screen: A startup button (e.g., "Start Game").
 - Character Select Menu: Essential. Showcases 3+ characters with their names, portraits, playstyle tags (e.g., balanced, tricky but slightly stronger), and move lists. Animated transitions for selection.
 - In-Game UI: Health bars, Ki meters, combo indicators.
 - Replay Screen: Essential.

5. Tools & Learning Strategy

- Coding Language: Exclusively GDScript, which is very beginner-friendly and similar to Python. You only need to learn specific parts for game mechanics.
- Graphics & Animations: No custom drawing or animating needed.
 - Characters: Use free pre-made 3D anime-style models from sources like Mixamo (rigged characters with animations), Godot Asset Library, or OpenGameArt. Models can be recolored in Godot (e.g., white hair for Roku, black/purple for Frieza Black, green armor for Vegeta).
 - Animations: Use pre-made animations (e.g., from Mixamo) for attacks and supers. Godot's AnimationPlayer will be used to keyframe poses, camera zooms, and particle effects.
 - Stage: Build with free cracked stone textures (OpenGameArt, Godot Asset Library), cosmic skybox (Godot Asset Library), and simple 3D meshes or Particles2D/3D for debris.
- Sound: Use free anime-style SFX and music from online sources like Freesound.org or Zapsplat.
- Build & Sharing: The final game can be built into a shareable file.

6. 20-Hour Feasibility & Key to Success

Achieving a "full game" in 20 hours is possible, provided a tight scope and smart

use of resources:

- Leverage Free Assets: Relying heavily on pre-made 3D models, textures, and particle packs eliminates time-consuming art creation.
- Simplify GDScript: The Python-like syntax is beginner-friendly, and concise, reusable scripts will be prioritised.
- Cutsцен Shortcuts: Use Godot's AnimationPlayer and pre-made animations for quick, dramatic cinematic sequences instead of hand-drawn scenes.
- Prioritise Polish: Focus on cel-shading, bloom, particle effects, and animated text to create a high-quality feel without complex underlying code.
- Stick to Scope: Limit to 3 characters and 1 stage.
- Reuse Logic: Core systems (combat, animation, UI) will be reused across characters.
- Test Early: Regular testing is crucial for catching bugs within the tight timeline.